End-to-End GCP Clinical Data Engineering Architecture

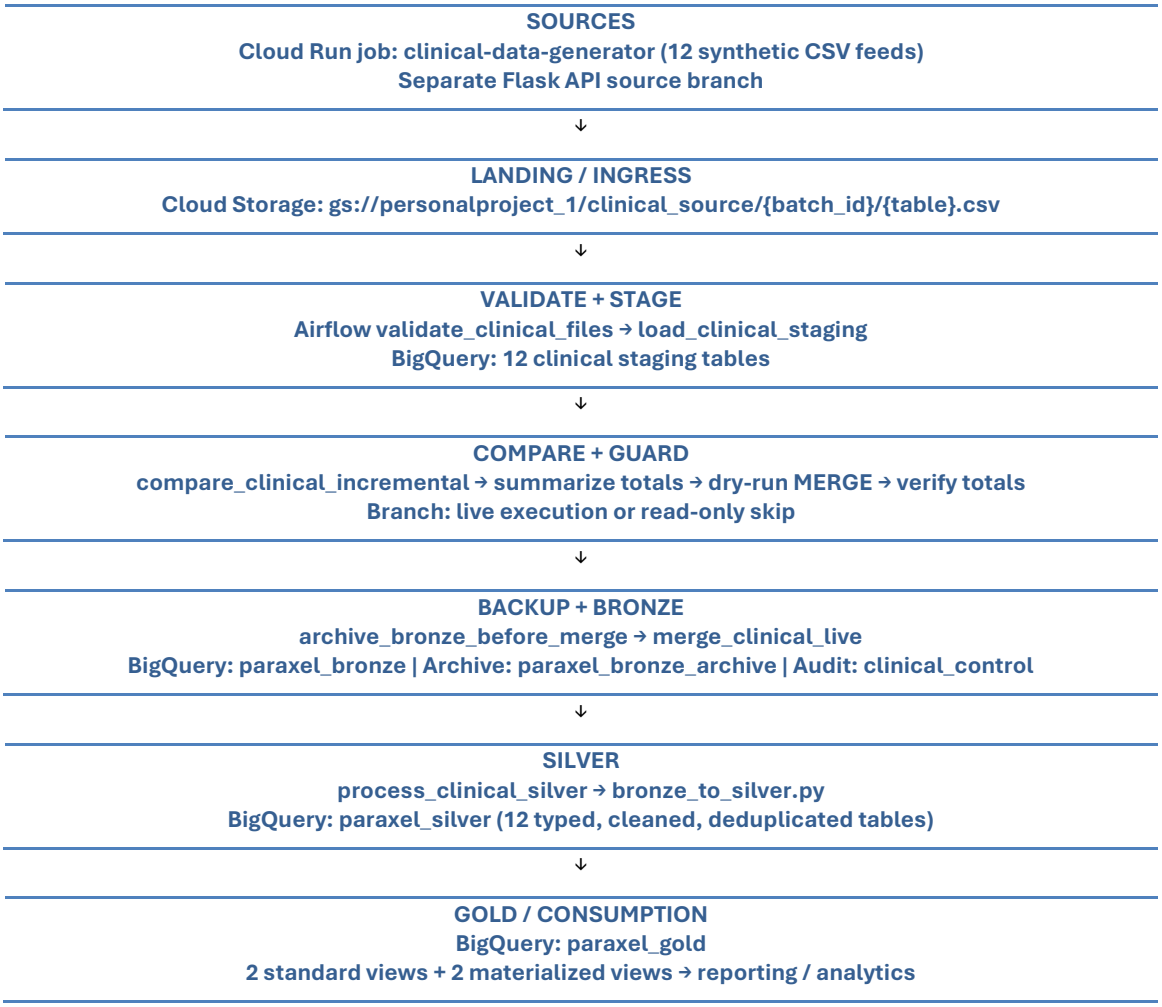
Actual implementation | Technology • Work performed • How it works • Outcome

Project: vaarahigcpproject | Orchestration: Cloud Composer (paraxelairflow) | Region: us-central1 | BigQuery location: US

1. Executive architecture summary

We implemented a batch-oriented clinical data pipeline that generates 12 related synthetic clinical CSV feeds, lands them in Cloud Storage, validates and loads them to BigQuery staging, compares them with Bronze records, protects Bronze with a pre-merge archive, applies guarded incremental MERGEs, transforms the merged Bronze data into typed Silver tables, and exposes Gold reporting views. Cloud Composer / Apache Airflow coordinates dependencies, branch modes, retries and audit steps. A separate Flask API ingestion branch is retained in the same DAG.

2. Architecture diagram — actual flow



3. What we did at each stage

1. Source generation

Technology: Cloud Run Jobs; Python/Faker; Flask API (separate branch)

What we did: Created a Cloud Run job named clinical-data-generator to generate 12 related clinical CSV files. Retained the existing Flask API branch for separate API ingestion.

How we achieved it: Airflow invokes CloudRunExecuteJobOperator with a run-specific BATCH_ID; the generator writes batch-specific objects to GCS.

What we achieved: Repeatable synthetic source feeds; 12 clinical entities available per generated batch.

2. Landing / ingress

Technology: Google Cloud Storage (GCS)

What we did: Stored the source CSV files before warehouse ingestion, organized by batch ID and table.

How we achieved it: Objects follow gs://personalproject_1/clinical_source/{batch_id}/{table}.csv. This separates file arrival from downstream transformations.

What we achieved: Traceable batch input and ability to process a named existing batch.

3. File validation

Technology: Cloud Composer / Apache Airflow; PythonOperator

What we did: Checked that the expected clinical input files are present before staging loads.

How we achieved it: validate_clinical_files receives the batch ID; normal and existing-batch routes converge on validation.

What we achieved: Avoids proceeding to staging when required files are unavailable.

4. BigQuery staging

Technology: BigQuery; Python client / SQL

What we did: Loaded all 12 source CSVs into corresponding clinical staging tables.

How we achieved it: load_clinical_staging is downstream of validation; staging preserves a comparison-ready representation of incoming batch data.

What we achieved: Earlier verified batch: 53,500 rows across 12 files/tables.

5. Incremental comparison

Technology: BigQuery SQL; Airflow XCom for small result summaries

What we did: Classified staged rows as new, changed, or unchanged relative to Bronze.

How we achieved it: compare_clinical_incremental returns per-table counts; summarize_incremental_comparison aggregates counts.

What we achieved: Provides an explainable delta before any live change.

6. Dry-run safety gate

Technology: Airflow PythonOperator / BranchPythonOperator; BigQuery

What we did: Ran dry-run logic and checked that its totals matched comparison totals.

How we achieved it: merge_clinical_dry_run → verify_merge_dry_run_totals → choose_merge_execution. Test mode branches to skip_live_merge.

What we achieved: Prevents live MERGE when comparison and dry-run totals disagree; supports read-only testing.

7. Bronze backup

Technology: BigQuery archive dataset; GCS archive (project archive assets)

What we did: Preserved a pre-merge Bronze baseline for recovery and auditability.

How we achieved it: archive_bronze_before_merge runs before merge_clinical_live; archive tables in paraxel_bronze_archive; archive GCS bucket is paraxel-clinical-archive-vaarahi.

What we achieved: Previously observed backup captured 55,105 pre-merge Bronze rows.

8. Bronze incremental MERGE

Technology: BigQuery MERGE; PythonOperator; audit table

What we did: Inserted new records and updated changed records while leaving unchanged records untouched.

How we achieved it: merge_clinical_incremental uses per-table primary keys, batch ID and backup suffix; writes execution results to clinical_control.incremental_merge_audit.

What we achieved: One successful live run: 1,652 new; 3,858 changed; 49,595 unchanged; Bronze 55,105 → 56,757 rows.

9. Silver curation

Technology: BigQuery SQL MERGE; google-cloud-bigquery Python client

What we did: Cleaned strings, converted source types, removed duplicate source keys and upserted all 12 tables.

How we achieved it: bronze_to_silver.py reads existing Silver schemas; TRIM + NULLIF + SAFE_CAST; ROW_NUMBER() deduplication; MERGE updates only changed values.

What we achieved: process_clinical_silver succeeded for all 12 clinical tables after fixing FLOAT → FLOAT64 cast.

10. Gold reporting

Technology: BigQuery standard views and materialized views

What we did: Exposed curated clinical summaries for downstream analytics without rebuilding raw transformations.

How we achieved it: Standard views reference Silver; existing materialized views support reusable aggregated reporting.

What we achieved: 4 Gold objects created: 2 standard views and 2 materialized views.

11. Orchestration & operations

Technology: Cloud Composer (Apache Airflow), Cloud Logging, gcloud SDK

What we did: Automated task dependencies, modes, retries, error reporting and operator execution.

How we achieved it: clinical_data_pipeline; hourly schedule 0 * * * *; max_active_runs=1; retries=2 with 5-minute delay; normal, test_existing_batch and confirmed apply_existing_batch modes.

What we achieved: End-to-end orchestration built; observed successful Silver task and historical live Bronze MERGE.

4. The 12 clinical tables

adverse_events • clinical_sites • clinical_studies • investigators • lab_results • medical_history • medications • patient_enrollments • patient_visits • patients • study_treatments • vital_signs

5. Gold reporting assets already implemented

Object	Type	Use / source
v_study_enrollment_summary	Standard view	Silver clinical_studies + patient_enrollments + clinical_sites; study-wise patient and site counts
v_treatment_analysis	Standard view	Silver study_treatments + patient_enrollments; treatment-arm patient counts
mv_daily_event_monitoring	Materialized view	Existing event monitoring aggregate; definition not separately verified here
mv_study_enrollment_status	Materialized view	Existing enrollment status aggregate; definition not separately verified here

6. Main DAG paths and operating modes

Mode	What happens	Safety
normal	Cloud Run generator + separate Flask API branch; 12-table staging → compare → dry-run → archive → live Bronze MERGE → Silver MERGE	Runs writes automatically; schedule is hourly when unpaused
test_existing_batch	Uses supplied batch ID; validates/compares and performs dry-run	Skips Bronze live MERGE and Silver write branch
apply_existing_batch	Processes supplied batch ID with live execution	Requires confirm_live_merge=true

Separate Flask branch: trigger_flask_api → read_pipeline_config → read_extra_config → check_gcs_file → audit_start → load_gcs_to_bronze → transform_bronze_to_silver → audit_success. This is distinct from the 12-table clinical Silver task.

7. Actual results versus unverified production claims

Evidence-backed outcome	Scope / caveat
12 clinical CSV feeds and 12 Silver tables	Implemented in synthetic clinical data project
53,500 staged rows	Previously observed batch, not a measured daily production volume
1,652 new / 3,858 changed / 49,595 unchanged	One observed successful live Bronze MERGE; not a universal batch profile
55,105 → 56,757 Bronze rows	One observed live run; total across tracked clinical tables
Silver MERGE success across all 12 tables	Observed Airflow task logs after FLOAT64 correction
Two standard Gold views and two materialized views	Created; standard-view definitions confirmed against Silver
Cost savings, latency SLA, uptime, security masking, Spark OOM remediation	Not measured or not evidenced for this implemented workflow; do not present as achievements

8. Key design decisions and limitations

- BigQuery handles the implemented 12-table clinical transformations; Dataproc/PySpark appeared in earlier separate practice work, not as the current 12-table Silver engine.
- Bronze preserves source-oriented values; Silver dynamically follows target table schema, using SAFE_CAST. Malformed nonblank values can become NULL; stronger rejection/quarantine rules are a production enhancement.
- Silver deduplication uses ROW_NUMBER() over the primary key with a deterministic row-content tie-breaker. It does not implement business latest-update-wins unless a trustworthy source update timestamp is added.
- BigQuery MERGE is atomic per table; a failure partway through the 12-table loop may leave earlier tables committed. Reruns are designed to be idempotent for stable input and unique keys.
- Standard Gold views query current Silver data; materialized view freshness depends on BigQuery refresh behavior. No explicit Gold refresh Airflow task was implemented.
- The final overall DAG-run state was not independently checked after the successful Silver task; the screenshot supports the Silver task success, not a separately verified end-task outcome.
- Least-privilege IAM, masking, partition/clustering design and regulatory compliance require separate evidence; they should not be claimed as completed merely because they are common GCP best practices.

9. Interview-ready 60–90 second explanation

"I developed a batch clinical data engineering pipeline on Google Cloud. A Cloud Run job generates 12 related clinical CSV feeds and lands them in batch-specific Cloud Storage paths. Cloud Composer orchestrates file validation, loading to BigQuery staging, incremental comparison against Bronze, dry-run verification, a pre-merge archive, and a guarded live BigQuery MERGE. We capture audit results and support read-only testing separately from confirmed live processing. After Bronze updates, a reusable Python/BigQuery module cleans and casts source values, deduplicates by primary key, and incrementally merges data into 12 typed Silver tables. The Gold layer exposes standard and materialized views for study enrollment, treatment and event reporting. In one successful run, we identified 1,652 new, 3,858 changed and 49,595 unchanged records; the Silver processing task later completed successfully for all 12 tables. I also resolved a BigQuery FLOAT-versus-FLOAT64 casting issue during integration."

Project: Clinical Trial Data Ingestion and Processing Pipeline on GCP

Tell me About your Project for 2 mins:

I worked on a **healthcare data engineering project focused on clinical trial data processing**, where our main objective was to automate the ingestion, transformation, and reporting of clinical study and patient-related information.

We followed the **Medallion Architecture — Bronze, Silver, and Gold**, using Google Cloud Platform.

Our data included clinical studies, patients, patient enrollments, investigators, clinical sites, lab results, medications, adverse events, and other clinical information. We worked with **12 interconnected clinical tables**.

For ingestion, we used **Python, Flask API, Cloud Run, and Google Cloud Storage**. Data arrived as CSV files in our GCS landing zone, and we used **Cloud Composer with Apache Airflow** to orchestrate the entire pipeline.

From GCS, we loaded the data into **BigQuery staging tables**, where we performed incremental comparisons against the existing Bronze data.

We classified incoming records into **new, changed, and unchanged records** using primary keys and field-level comparisons.

Before updating Bronze, we created backups for recovery. Then we used **BigQuery MERGE statements** to insert new records and update existing records without unnecessarily processing unchanged data.

In the **Silver layer**, we performed data cleansing, datatype conversions using `SAFE_CAST`, NULL handling, and deduplication using SQL window functions. We also used incremental MERGE operations to maintain the curated tables.

Finally, in the **Gold layer**, we created standard and materialized views for study enrollment reporting, treatment analysis, and clinical event monitoring.

We implemented audit logging, retries, dependency management, and controlled execution through Airflow.

Overall, the project automated the end-to-end movement of clinical data from source systems to business-ready reporting layers, while improving data consistency, traceability, and maintainability.

Project: End-to-End Clinical Data Engineering Pipeline on Google Cloud Platform

Explain your Project for 5-7 mins

1. Project Introduction and Business Requirement

I worked on a healthcare domain data engineering project focused on clinical trial data ingestion, processing, and analytics.

The main business requirement was to build an automated and reliable data pipeline to process clinical study information collected from different sources.

The data included patient details, clinical studies, patient enrollments, investigators, clinical sites, laboratory results, medications, adverse events, medical history, patient visits, and treatment information.

We worked with **12 related clinical tables**, and these datasets were interconnected through identifiers such as `patient_id`, `study_id`, `site_id`, and `enrollment_id`.

The business needed consistent, cleaned, and structured data for monitoring study progress, patient enrollment, treatment analysis, and clinical event reporting.

To achieve this, we designed a pipeline using **Google Cloud Platform and Medallion Architecture**, consisting of Bronze, Silver, and Gold layers.

2. Data Ingestion — Python, Flask API, Cloud Run and GCS

Our first stage was data ingestion.

We implemented two ingestion workflows. One used a Python Flask API, and the other used a Python-based clinical data generator deployed as a **Cloud Run Job**.

The Cloud Run Job generated 12 CSV files representing the clinical datasets and stored them in **Google Cloud Storage**, which acted as our landing zone.

Each batch was organized using a unique batch identifier. This helped us track files belonging to a particular pipeline execution.

We used **Apache Airflow running on Cloud Composer** to trigger the Cloud Run Job and manage downstream processing.

Before loading the files into BigQuery, we validated that the expected clinical files were available.

We chose GCS because it provides scalable object storage and separates source ingestion from warehouse processing.

3. BigQuery Staging and Incremental Comparison

Once the files arrived in GCS, our Airflow DAG loaded them into 12 BigQuery staging tables.

The staging layer temporarily held the latest incoming batch.

One of the important requirements was to avoid blindly inserting all records into Bronze during every execution.

For this, we implemented an incremental comparison process.

We compared incoming staging records with existing Bronze records using the primary key defined for each table.

The comparison identified three categories:

- **New records:** Primary keys not present in Bronze.
- **Changed records:** Existing primary keys with differences in source fields.
- **Unchanged records:** Existing primary keys with matching values.

For example, in one of our successful incremental batches, we processed **53,500 incoming records** and identified approximately **1,652 new records, 3,858 changed records, and 49,595 unchanged records**.

This approach allowed us to update only the necessary records while maintaining historical continuity in the Bronze dataset.

4. Bronze Layer — Raw Storage, Backup and MERGE

In the Bronze layer, we maintained the raw clinical data in BigQuery.

We preserved incoming source fields primarily as STRING columns, so the Bronze layer remained close to the original source format.

Before performing the actual incremental MERGE, we ran a dry-run comparison to verify that the expected new, changed, and unchanged record counts matched our comparison results.

We also created a backup of the existing Bronze tables before modifying them.

For this, we used a separate BigQuery archive dataset and a GCS archive location for recovery-related storage.

After the backup succeeded, we executed **BigQuery MERGE statements**.

The MERGE logic inserted records when the primary key was new and updated records when an existing primary key had changed.

Unchanged records did not require unnecessary updates.

We also maintained audit information in BigQuery, including batch IDs, table names, record counts, execution status, timestamps, and error details.

This provided traceability and helped us identify which tables succeeded or failed during processing.

5. Silver Layer — Data Cleansing and Transformations

After the Bronze MERGE completed successfully, Airflow triggered our Silver processing module.

The Silver layer contained the same 12 clinical datasets, but with cleaned and properly typed columns.

We implemented these transformations using **Python and BigQuery SQL**.

First, we applied `TRIM` to remove unwanted spaces from source values.

We converted blank strings into NULL values using `NULLIF`.

Then we used `SAFE_CAST` to convert raw STRING fields into the required target datatypes, such as DATE, NUMERIC, FLOAT64, and BOOLEAN.

For deduplication, we used SQL window functions, particularly `ROW_NUMBER()`, to retain one record per primary key.

An important design decision was to read the existing Silver table schemas dynamically from BigQuery metadata.

This meant we did not have to hardcode every column datatype separately for all 12 tables.

After cleansing and datatype conversion, we used **BigQuery MERGE** to insert new records and update existing records in Silver.

This helped us maintain curated datasets that were suitable for downstream analytics.

6. Gold Layer — Business Reporting and Analytics

Once the Silver layer was ready, we used the Gold layer to expose business-oriented datasets.

We created standard BigQuery views and materialized views.

Our standard views included:

- `v_study_enrollment_summary`, which combined clinical studies, patient enrollments, and clinical sites to report study-level patient and site counts.
- `v_treatment_analysis`, which combined treatment assignments and patient enrollment data to calculate patient counts by treatment arm.

We also created materialized views for study enrollment status and daily clinical event monitoring.

The standard views reference the Silver tables directly, so they reflect updated Silver data when queried.

Materialized views provide precomputed results where their refresh behavior and query definitions support it.

The Gold layer simplified reporting by exposing business-focused results rather than requiring downstream consumers to repeatedly join multiple clinical tables.

7. Workflow Orchestration — Apache Airflow and Cloud Composer

We used **Cloud Composer, Google's managed Apache Airflow service**, to orchestrate the complete pipeline.

Our main DAG was named `clinical_data_pipeline`.

The DAG controlled the dependencies between ingestion, validation, staging, incremental comparison, Bronze backup, live MERGE, and Silver processing.

We scheduled the DAG hourly and configured retries with a five-minute retry delay.

We also used Airflow branching to support different execution modes.

The normal mode executed the ingestion and incremental processing workflow.

The read-only test mode allowed us to validate an existing batch without modifying Bronze.

For controlled manual updates, we also implemented an execution mode requiring explicit confirmation before performing a live MERGE.

We configured `max_active_runs=1` to prevent overlapping DAG runs.

For smaller task results and control information, we used Airflow's task communication mechanisms, while the actual clinical data remained in GCS and BigQuery.

8. Error Handling and Troubleshooting

During implementation, we encountered several practical engineering challenges.

One issue was that some Airflow runs remained queued because the DAG was paused or execution capacity was constrained by the active-run limit.

We also encountered a datatype conversion error during Silver processing.

BigQuery rejected a generated SQL statement because the datatype was written as `FLOAT` instead of `FLOAT64`.

We identified the issue through Cloud Composer task logs, corrected the datatype mapping in our Python module, uploaded the updated code, and reran only the failed Silver task.

The remaining tables completed successfully without requiring us to rerun the complete ingestion pipeline.

We also designed Bronze backups and audit logging to improve recoverability and traceability.

9. Final Outcome

By the end of the project, we had implemented an automated clinical data pipeline using **Python, Flask, Cloud Run, GCS, Cloud Composer, Apache Airflow, and BigQuery**.

We successfully processed 12 interconnected clinical datasets through staging, Bronze, and Silver layers.

We implemented incremental comparison, MERGE-based updates, audit logging, backups, schema-driven transformations, deduplication, and business reporting views.

The key outcome was a reusable, maintainable data engineering workflow that transformed raw clinical data into structured and business-ready datasets.

My main responsibilities included developing the ingestion and transformation logic, implementing BigQuery incremental processing, configuring Airflow orchestration, handling failures, and supporting the end-to-end pipeline execution.

⇨ Role and responsibilities

I worked as a **Data Engineer**, mainly responsible for developing and maintaining data ingestion, transformation, and orchestration pipelines on Google Cloud Platform.

My responsibilities included:

- **Data Ingestion:** Developed Python-based ingestion workflows using Flask API, Cloud Run, and Google Cloud Storage to bring clinical data into our landing zone.
- **BigQuery Processing:** Implemented staging-to-Bronze incremental processing using primary-key comparisons and BigQuery `MERGE` statements to handle new and changed records.
- **Data Transformation:** Developed Bronze-to-Silver transformations using BigQuery SQL, including `SAFE_CAST`, NULL handling, data cleansing, and deduplication using `ROW_NUMBER()`.
- **Workflow Orchestration:** Developed and maintained Apache Airflow DAGs in Cloud Composer, configuring dependencies, retries, scheduling, and failure handling.
- **Data Quality and Monitoring:** Implemented file validation, audit logging, incremental reconciliation, and Bronze backups to improve data reliability.
- **Reporting:** Supported Gold-layer views and materialized views for study enrollment, treatment analysis, and clinical event monitoring.

My primary focus was to make sure clinical data moved reliably from source ingestion to business-ready datasets while maintaining data quality and traceability.

⇨ Challenges You faced-1

Context: One of the major challenges in our clinical data pipeline was handling incremental loads efficiently. Each incoming batch contained a combination of new, modified, and unchanged clinical records.

If we processed every record as a new insert, it could create duplicate records and unnecessary updates in BigQuery.

Action: To solve this, I implemented an incremental comparison process between the staging and Bronze tables using primary keys and field-level comparisons.

We classified records as new, changed, or unchanged. Before performing the actual updates, we executed a dry run to reconcile the expected counts and created a Bronze backup.

Then we used BigQuery `MERGE` statements to insert new records and update only the modified records. We also maintained audit records to track processing results.

Result: In one successful batch of 53,500 incoming records, we identified **1,652 new, 3,858 changed, and 49,595 unchanged records**.

This allowed us to avoid unnecessary updates for approximately 92.7% of the incoming records while maintaining consistent Bronze data.

⇨ Challenges You faced-2

Context: During our Bronze-to-Silver transformation, we encountered a datatype conversion failure while processing clinical laboratory results.

Our Bronze tables stored raw values as STRING, whereas the Silver tables required appropriate datatypes such as DATE, NUMERIC, FLOAT64, and BOOLEAN.

Action: I investigated the Cloud Composer task logs and identified that our dynamically generated BigQuery SQL was using `FLOAT` instead of the supported `FLOAT64` datatype.

I corrected the datatype mapping in our Python transformation module, uploaded the updated code to Composer, and reran the failed Silver task.

Since the transformation used BigQuery `MERGE`, previously processed tables could be safely processed again without creating duplicate primary keys.

Result: The corrected task successfully processed all **12 clinical Silver tables**. We recovered without rerunning the complete ingestion and Bronze pipeline.

This experience helped us improve the reliability of our schema-driven transformation logic.

⇨ Challenges You faced-3

Context: During pipeline execution, we noticed that some Cloud Composer DAG runs remained in the `queued` state, even after being triggered successfully.

We also needed to prevent overlapping runs from modifying the same clinical tables simultaneously.

Action: I investigated the Airflow Grid and DAG configuration. We identified that the DAG pause state and our `max_active_runs=1` configuration affected when new runs could start.

We unpaused the DAG when execution was required and avoided triggering unnecessary duplicate runs.

We also configured task retries, execution dependencies, and branching to separate read-only tests from live MERGE processing.

Result: The queued DAG progressed to execution, and we successfully completed the clinical data processing workflow.

This helped us understand how Airflow scheduling, branching, retries, and concurrency controls affect production-style pipelines.

⇨ Challenges You faced-4

Context: Clinical datasets contain interconnected study and patient information. An incorrect incremental MERGE could overwrite existing values or introduce inconsistent records.

Action: We designed a controlled MERGE workflow with multiple safeguards.

First, we compared staging and Bronze records to identify new, changed, and unchanged records. Then we executed a MERGE dry run and reconciled the results against the comparison totals.

Before live updates, we created Bronze backups and recorded processing results in BigQuery audit tables.

We also implemented separate Airflow execution modes for read-only testing and explicitly confirmed manual live updates.

Result: We successfully executed the incremental MERGE with backup and audit records, making the pipeline easier to trace and providing a recovery source if incorrect data was written.

The key lesson was that recovery and validation should be built into the pipeline design rather than handled only after a failure.